

WAP to Implement Singly Linked List with following operations

- a) Create a linked list.
- b) Deletion of first element, specified element and last element in the list.
- c) Display the contents of the linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
struct node *head = NULL;
void createList(int n) {
    struct node *newNode, *temp;
    int data, i;
    if (n <= 0) {
        printf("Number of nodes should be greater than 0.\n");
        return;
    }
    for (i = 1; i <= n; i++) {
        newNode = (struct node*)malloc(sizeof(struct node));
        if (newNode == NULL) {
            printf("Memory allocation failed.\n");
            return;
        }
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        newNode->data = data;
        newNode->next = NULL;
        if (head == NULL) {
            head = newNode;
        } else {
            temp->next = newNode;
        }
        temp = newNode;
    }
    printf("\nLinked list created successfully.\n");
}
void deletefirst(){
    struct node *temp;
    if(head==NULL){
        printf("list is empty.nothing to delete.\n");
```

```

        return;
    }
    temp=head;
    head=head->next;
    printf("deleted element:%d\n",temp->data);
    free(temp);
}
void deletelast() {
    struct node *temp, *prev;
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted element: %d\n", head->data);
        free(head);
        head = NULL;
        return;
    }
    temp = head;
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }
    printf("Deleted element: %d\n", temp->data);
    prev->next = NULL;
    free(temp);
}
void deletespecific(int value) {
    struct node *temp = head,*prev = NULL;
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
    if (head->data == value) {
        temp = head;
        head = head->next;
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }

```

```

    }
    if (temp == NULL) {
        printf("Element %d not found in the list.\n", value);
        return;
    }
    prev->next = temp->next;
    printf("Deleted element: %d\n", temp->data);
    free(temp);
}

void displaylist() {
    struct node *temp = head;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("\nLinked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, n, value;

    while (1) {
        printf("\n--- Singly Linked List Operations ---\n1. Create Linked List\n2. delete first\n3.delete specific\n4.delete end\n5. Display List\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter number of nodes: ");
                scanf("%d", &n);
                createList(n);
                break;
            case 2:
                deletefirst();
                break;
            case 3:
                printf("Enter value to delete: ");
                scanf("%d", &value);
                deletespecific(value);
                break;

```

```
case 4:
    deletelast();
    break;
case 5:
    displaylist();
    break;
case 6:
    printf("Exiting...\n");
    exit(0);
default:
    printf("Invalid choice. Try again.\n");
}
}
return 0;
}
```

--- Singly Linked List Operations ---

1. Create Linked List
2. delete first
- 3.delete specific
- 4.delete end
5. Display List
6. Exit

Enter your choice: 1

Enter number of nodes: 5

Enter data for node 1: 10

Enter data for node 2: 20

Enter data for node 3: 30

Enter data for node 4: 40

Enter data for node 5: 50

Linked list created successfully.

--- Singly Linked List Operations ---

1. Create Linked List
2. delete first
- 3.delete specific
- 4.delete end
5. Display List
6. Exit

Enter your choice: 2

deleted element:10

--- Singly Linked List Operations ---

1. Create Linked List
2. delete first
- 3.delete specific
- 4.delete end
5. Display List
6. Exit

Enter your choice: 3

Enter value to delete: 40

Deleted element: 40

--- Singly Linked List Operations ---

1. Create Linked List
2. delete first
- 3.delete specific
- 4.delete end
5. Display List
6. Exit

Enter your choice: 4

Deleted element: 50

--- Singly Linked List Operations ---

1. Create Linked List

2. delete first

3.delete specific

4.delete end

5. Display List

6. Exit

Enter your choice: 5

Linked List: 20 -> 30 -> NULL

--- Singly Linked List Operations ---

1. Create Linked List

2. delete first

3.delete specific

4.delete end

5. Display List

6. Exit

Enter your choice: 6

Exiting...

Process returned 0 (0x0) execution time : 46.144 s

Press any key to continue.

|